

STRIDE

Supply chain Tracking, Routing, Inventory & Distribution Engine —
Automation & n8n Workflow Architecture

Abhinand Baiju Smitha

Final Year Project — Presidency University, Department of Computer
Science and Engineering

Contents

1 Purpose & Scope	3
2 Why Event-Driven Automation Fits This Domain	4
2.1 The Coordination Problem	4
2.2 Why n8n Specifically	4
2.3 Where This Sits in the Layered Architecture	4
3 Event Catalog	6
3.1 Payload Schemas	6
3.1.1 order.allocated	6
3.1.2 stock.below_threshold	7
3.1.3 order.shipped	7
3.1.4 stock.backordered	7
4 WebhookEventPublisher Design	8
4.1 Responsibility & Placement	8
4.2 Why AFTER_COMMIT, Not a Direct Call	9
4.3 Reliability: At-Least-Once, Not Exactly-Once	9
5 Security: Signing Webhook Payloads	10
5.1 HMAC Signing Scheme	10
5.2 Why This Matters Specifically Here	10
6 n8n Workflow Specifications	11
6.1 Workflow 1: order.allocated	11
6.2 Workflow 2: stock.below_threshold	11
6.3 Workflow 3: order.shipped	12
6.4 Workflow 4: stock.backordered	12
6.5 Workflow 5: order.cancelled	12
7 Idempotency on the n8n Side	13

8 Reliability Hardening (Future Work): Outbox Pattern	14
9 Docker Compose Addition	15
10 Testing Strategy for the Automation Layer	16
10.1 Unit Test — Signature Generation	16
10.2 Integration Test — Failure Isolation	16
10.3 Manual/Exploratory Verification (for the demo)	16
11 Decision Note: Twilio Integration Point	17
12 Updated Build Plan Addendum	18

1 Purpose & Scope

The core specification and architecture reference describe STRIDE as a system of record: it decides *how* an order is fulfilled and persists that decision correctly under concurrency. What it does not do on its own is **coordinate what happens next across other systems and people** — notifying a supplier, escalating a stockout, emailing a customer, alerting a warehouse team on Slack. In a real supply chain operation, that coordination work is the majority of day-to-day system behavior, and doing it well is what separates a demo from something that looks like it could actually run a warehouse.

This document specifies the **automation layer**: how STRIDE (Spring Boot) emits events at meaningful moments, and how **n8n** consumes those events to orchestrate downstream workflows — notifications, purchase-order drafts, escalations — without any of that orchestration logic living inside the core backend.

This is an **addition**, not a replacement, to the architecture described in the other two documents. Everything in `STRIDE_Project_Specification.pdf` and `STRIDE_Architecture_Reference.pdf` still applies; this document only adds the event-emission points and the workflows that consume them.

2 Why Event-Driven Automation Fits This Domain

2.1 The Coordination Problem

A supply chain platform sits at the intersection of several parties who don't share a database: customers, warehouse staff, suppliers, and finance. Every meaningful state change in STRIDE — an order gets allocated, a warehouse runs low on stock, a shipment goes out — needs to reach one or more of these parties, often through different channels (SMS, email, Slack, a supplier's own ordering portal). Hardcoding this fan-out into `OrderService` would violate the same Single Responsibility principle the core architecture is built around (main spec, Section 11.4) — `OrderService`'s job is to correctly place an order, not to know that Warehouse 3's manager prefers Slack and Supplier X needs an emailed purchase order.

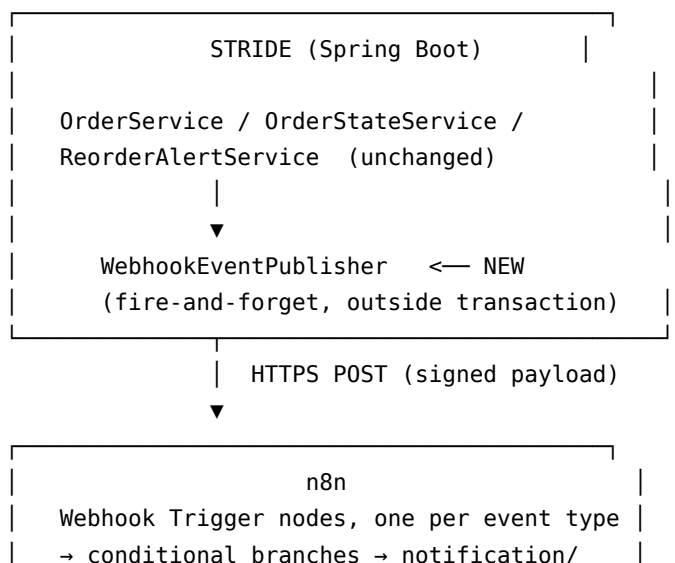
2.2 Why n8n Specifically

n8n is a workflow-automation tool that consumes webhooks and orchestrates multi-step logic (conditionals, branching, multiple destinations) without that logic being hardcoded into the emitting application. This is the correct architectural boundary:

- **Spring Boot's job:** decide *that* something happened, and emit a fact about it (an event) — nothing more.
- **n8n's job:** decide *what to do* about that fact — who to notify, through which channel, whether to escalate, whether to draft a purchase order.

This separation means adding a new notification channel, changing an escalation rule, or onboarding a new supplier's integration **never requires a Spring Boot code change or redeploy** — it's a workflow edit in n8n. That is precisely the kind of flexibility real supply chain operations need, since business rules around “who gets notified when” change far more often than the core fulfillment logic does.

2.3 Where This Sits in the Layered Architecture



| purchase-order/escalation nodes |

WebhookEventPublisher follows exactly the same architectural rule already established for TwilioSmsClient in the main spec (Section 18): it is called **outside** any @Transactional boundary, wraps its call in try/catch, and never throws upward. A slow or down n8n instance must never fail an order placement — this is non-negotiable and is elaborated in Section 6.

3 Event Catalog

Every event STRIDE emits, its trigger point (cross-referenced to the lifecycle traces in the architecture reference), and its payload shape.

Event name	Emitted from	Trigger point
order.allocated	OrderService.placeOrder	after successful commit, full coverage (Lifecycle A, after step 7)
order.partially_allocated	OrderService.placeOrder	after successful commit, backorder present (Lifecycle C)
order.shipped	OrderStateService	when the last allocation under an order transitions to SHIPPED (Lifecycle D, step 4)
order.cancelled	OrderStateService.cancel	after successful cancellation commit (Lifecycle E, step 4)
stock.below_threshold	ReorderAlertService	daily scheduled job, per warehouse with low stock (Lifecycle F)
stock.backordered	OrderService.placeOrder	when any line has backorderedQuantity > 0, emitted alongside order.partially_allocated but scoped per product for supplier-specific routing

3.1 Payload Schemas

3.1.1 order.allocated

```
{
  "eventType": "order.allocated",
  "eventId": "evt_9f2a...",
  "occurredAt": "2026-08-20T10:15:30Z",
  "orderId": "o-7712",
  "customerId": "c-4410",
  "lines": [
    { "productId": "p-9931", "quantityRequested": 40,
      "allocations": [
        { "warehouseId": "w-BLR-01", "quantityAllocated": 30 },
        { "warehouseId": "w-BLR-04", "quantityAllocated": 10 }
      ]
    }
  ]
}
```

```
]
}
```

3.1.2 stock.below_threshold

```
{
  "eventType": "stock.below_threshold",
  "eventId": "evt_1a7c...",
  "occurredAt": "2026-08-20T06:00:00Z",
  "warehouseId": "w-BLR-01",
  "warehouseName": "Bengaluru Central",
  "managerContacts": [ { "userId": "u-88", "phone": "+91...", "email": "..." } ],
  "items": [
    { "productId": "p-9931", "sku": "SKU-9931", "quantityRemaining": 4, "reorderThreshold": 10 }
  ]
}
```

3.1.3 order.shipped

```
{
  "eventType": "order.shipped",
  "eventId": "evt_5d31...",
  "occurredAt": "2026-08-21T14:02:00Z",
  "orderId": "o-7712",
  "customerId": "c-4410",
  "shipments": [
    { "warehouseId": "w-BLR-01", "productIds": ["p-9931"] },
    { "warehouseId": "w-BLR-04", "productIds": ["p-9931"] }
  ]
}
```

3.1.4 stock.backordered

```
{
  "eventType": "stock.backordered",
  "eventId": "evt_2f90...",
  "occurredAt": "2026-08-20T10:15:31Z",
  "orderId": "o-7712",
  "productId": "p-1042",
  "sku": "SKU-1042",
  "quantityBackordered": 15,
  "supplierContacts": [ { "supplierId": "s-12", "email": "supplier@example.com" } ]
}
```

Every payload includes a globally unique `eventId` — this is not decorative, it is the basis for idempotent handling on the n8n side (Section 7).

4 WebhookEventPublisher Design

4.1 Responsibility & Placement

Exactly one new class, placed alongside `TwilioSmsClient` in the `notification/` package (architecture reference, component inventory). It is called from the service layer **after** a transaction has committed, never from inside one — this uses Spring's `@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)` mechanism rather than a direct method call, which is the correct way to guarantee the webhook only fires for state that has actually, durably happened (never for a transaction that later rolls back).

```
// Step 1: OrderService publishes an in-process application event, NOT an HTTP call directly
@Service
public class OrderService {
    private final ApplicationEventPublisher applicationEvents;

    @Transactional
    @Retryable(retryFor = ObjectOptimisticLockingFailureException.class, maxAttempts = 3)
    public OrderResponse placeOrder(OrderRequest request) {
        // ... existing logic from Lifecycle A ...
        Order order = allocationExecutor.persistOrderWithAllocations(request, plan);
        applicationEvents.publishEvent(new OrderAllocatedEvent(order, plan)); // queued, not sent yet
        return OrderMapper.toResponse(order);
    }
}

// Step 2: a separate listener fires the actual webhook, ONLY after commit succeeds
@Component
public class WebhookEventPublisher {

    private final RestClient restClient; // Spring's RestClient, short timeout configured
    private final String n8nWebhookBaseUrl;
    private final String webhookSigningSecret;

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    @Async // fire-and-forget on a separate thread, never blocks the HTTP response to the client
    public void onOrderAllocated(OrderAllocatedEvent event) {
        send("order.allocated", buildPayload(event));
    }

    private void send(String eventType, Object payload) {
        try {
            String json = objectMapper.writeValueAsString(payload);
            String signature = hmacSha256(json, webhookSigningSecret);
            restClient.post()
                .uri(n8nWebhookBaseUrl + "/" + eventType)
                .header("X-STRIDE-Signature", signature)
                .contentType(MediaType.APPLICATION_JSON)
        }
    }
}
```



```

        .body(json)
        .retrieve()
        .toBodilessEntity();
    } catch (Exception e) {
        log.warn("Webhook delivery failed for event {}: {}", eventType, e.getMessage());
        // deliberately swallowed – see Section 6.2 for why, and Section 8 for the mitigation
    }
}
}

```

4.2 Why AFTER_COMMIT, Not a Direct Call

If `WebhookEventPublisher` were called directly from inside `OrderService.placeOrder` (as a normal method call, before the transaction commits), two things could go wrong: the transaction could later roll back (e.g., a downstream check fails) while n8n has already told a supplier an order was allocated — a false event; and the webhook HTTP call, if slow, would extend the database transaction’s duration, reintroducing exactly the connection-pool-starvation risk the architecture reference (Section 6) warns about for the Twilio integration. `@TransactionalEventListener(AFTER_COMMIT)` combined with `@Async` solves both: the event only fires once the data is durably true, and it fires on a separate thread so the client gets their HTTP response immediately without waiting on n8n at all.

4.3 Reliability: At-Least-Once, Not Exactly-Once

A webhook call can fail (n8n down, network blip) and is not retried indefinitely by `WebhookEventPublisher` itself (Section 6.2 explains why not blocking retries specifically). This means event delivery is **best-effort**, not guaranteed — an acceptable tradeoff for notification/orchestration events, but one that must be stated explicitly rather than assumed away. Section 8 (Outbox Pattern as a Future Hardening Step) describes how to upgrade this to guaranteed delivery if time allows.

5 Security: Signing Webhook Payloads

Because these webhooks may trigger real actions (drafting a purchase order, notifying a supplier), n8n must be able to verify a payload genuinely came from STRIDE, not a spoofed request hitting the same n8n webhook URL.

5.1 HMAC Signing Scheme

- STRIDE computes `HMAC-SHA256(payload_body, shared_secret)` and sends it as the `x-STRIDE-Signature` header (shown in Section 5's code).
- The n8n workflow's first node after the Webhook Trigger is a **Function node** that recomputes the same HMAC over the raw received body using the same shared secret (stored as an n8n credential, never hardcoded in the workflow JSON) and compares it to the header — mismatch short-circuits the workflow immediately (an IF node routing to a "reject" no-op branch).
- The shared secret is provisioned the same way `JWT_SECRET` and `TWILIO_AUTH_TOKEN` are (architecture reference, Section 7 — environment variable, `.env`-sourced, never committed).

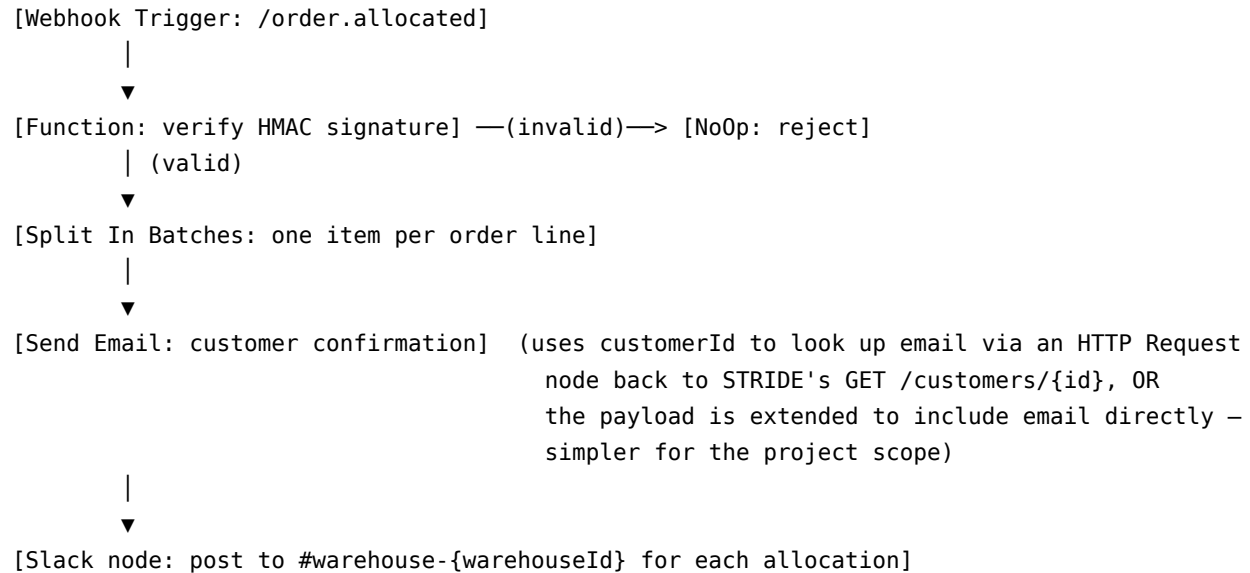
5.2 Why This Matters Specifically Here

Unlike the Twilio integration (STRIDE calls out, nothing calls in), webhook endpoints are **inbound-triggering** from n8n's perspective — anyone who discovers the n8n webhook URL could otherwise trigger a fake "order allocated" event and cause a real supplier notification or purchase-order draft to fire. Signing closes this gap and is not optional for anything beyond a local demo.

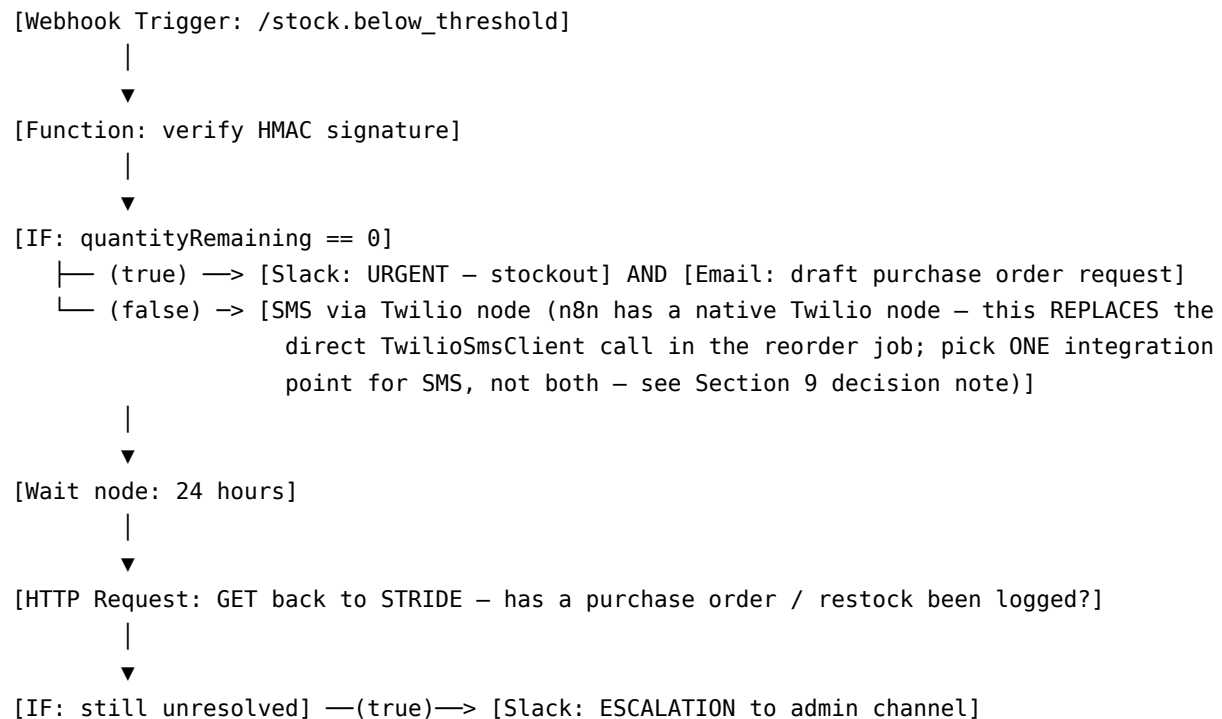
6 n8n Workflow Specifications

Each workflow below is described as a node sequence — implement each directly in the n8n editor as described.

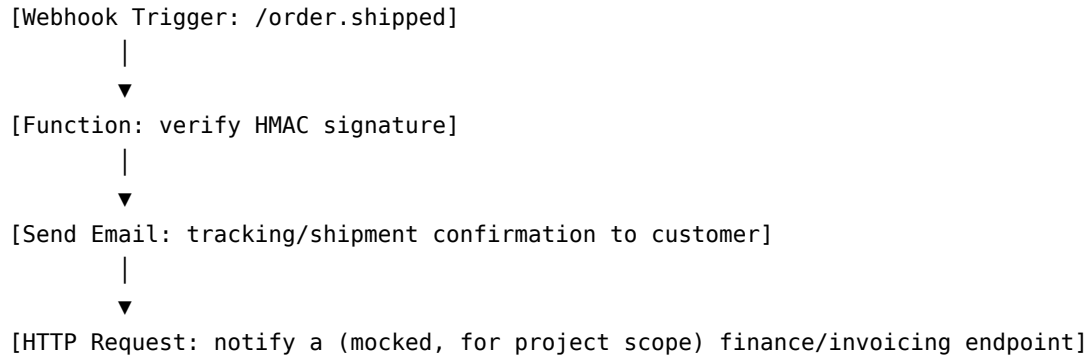
6.1 Workflow 1: order.allocated



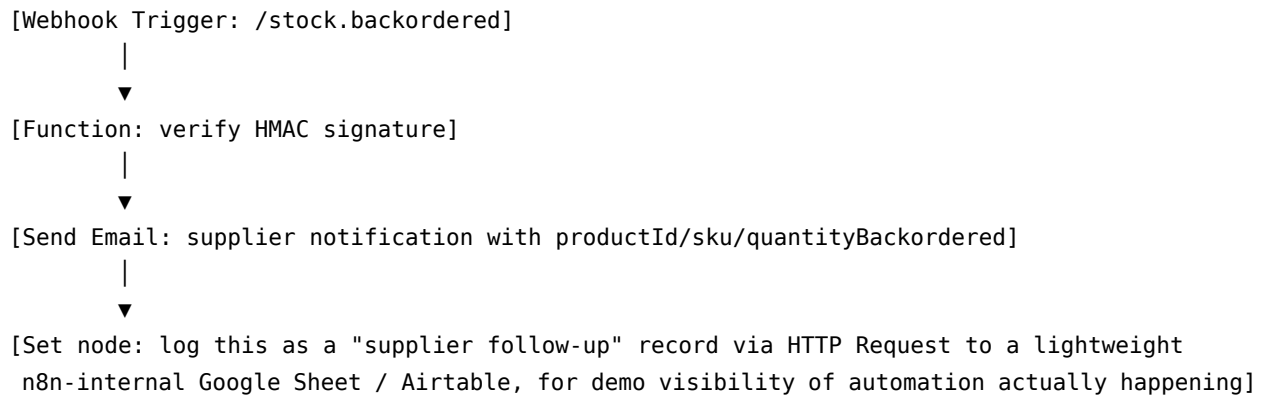
6.2 Workflow 2: stock.below_threshold



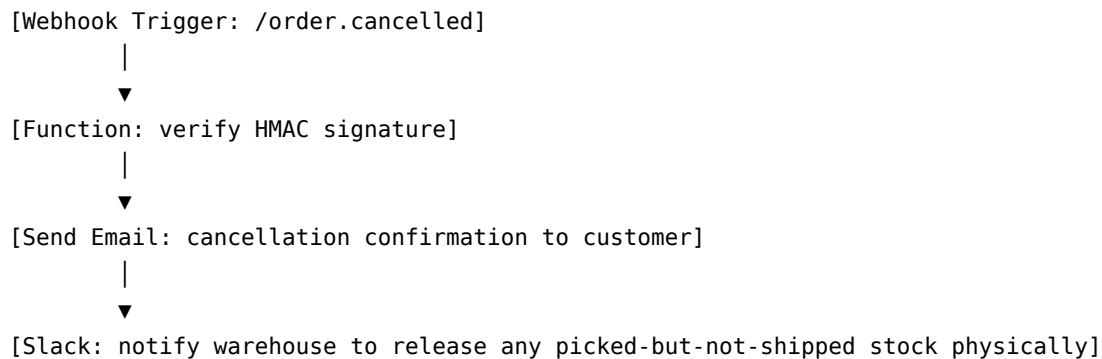
6.3 Workflow 3: order.shipped



6.4 Workflow 4: stock.backordered



6.5 Workflow 5: order.cancelled



7 Idempotency on the n8n Side

Because delivery is at-least-once (Section 5.2), the same event can, rarely, arrive twice (e.g., STRIDE retries a webhook call that actually succeeded but timed out on the response). Each workflow’s very first substantive node after signature verification should be an **idempotency check**: a Set/HTTP Request node that checks whether `eventId` has already been processed (a simple key-value check against an n8n data store, Airtable, or even a Google Sheet row lookup is sufficient for project scope), and short-circuits if so. This is a small addition per workflow but is worth stating explicitly in a viva — “what if the same event arrives twice” is a natural follow-up question once “what if it never arrives” has been asked.

8 Reliability Hardening (Future Work): Outbox Pattern

The baseline design (Section 5) is best-effort delivery — acceptable for a final-year project, but worth naming the proper fix as a documented extension, the same way the main spec documents the greedy-vs-ILP tradeoff explicitly rather than silently.

The Outbox Pattern: instead of firing the webhook directly from `WebhookEventPublisher`, write the event to an `outbox_event` table **within the same transaction** as the order placement (so it’s guaranteed to be recorded if and only if the order itself committed). A separate poller (`@Scheduled`, every few seconds) reads unprocessed outbox rows and attempts delivery, marking each as sent on success and retrying on failure — this upgrades delivery from best-effort to guaranteed-eventually, at the cost of a small amount of additional infrastructure. This is the correct production answer and is exactly the kind of “what would you do differently at scale” answer that demonstrates awareness beyond the project’s actual scope.

9 Docker Compose Addition

```
n8n:
  image: n8nio/n8n:latest
  ports: ["5678:5678"]
  environment:
    N8N_BASIC_AUTH_ACTIVE: "true"
    N8N_BASIC_AUTH_USER: ${N8N_USER}
    N8N_BASIC_AUTH_PASSWORD: ${N8N_PASSWORD}
    WEBHOOK_URL: "http://localhost:5678/"
  volumes: ["n8n_data:/home/node/.n8n"]
  depends_on:
    app:
      condition: service_started
```

Add `n8n_data` to the top-level `volumes` block alongside `pgdata` (main spec, Section 19.1). `app`'s environment needs one addition: `N8N_WEBHOOK_BASE_URL`: `http://n8n:5678/webhook`, using Docker's internal DNS the same way `postgres` is referenced (architecture reference, Section 7).

10 Testing Strategy for the Automation Layer

10.1 Unit Test — Signature Generation

```
@Test
void generatesConsistentHmacSignature() {
    String payload = "{\"eventType\":\"order.allocated\"}";
    String sig1 = WebhookSigner.sign(payload, "test-secret");
    String sig2 = WebhookSigner.sign(payload, "test-secret");
    assertThat(sig1).isEqualTo(sig2);

    String tamperedSig = WebhookSigner.sign(payload + "x", "test-secret");
    assertThat(tamperedSig).isNotEqualTo(sig1);
}
```

10.2 Integration Test — Failure Isolation

The single most important test for this layer, directly proving the architectural claim in Section 3 (“a slow or down n8n instance must never fail an order placement”):

```
@Test
void orderPlacementSucceedsEvenWhenWebhookEndpointIsDown() throws Exception {
    // n8nWebhookBaseUrl points to a port nothing is listening on
    seedWarehouseWithStock(WAREHOUSE_1, PRODUCT_A, 50);

    mockMvc.perform(post("/api/v1/orders")
        .contentType(APPLICATION_JSON)
        .content(orderRequestJson(PRODUCT_A, 20)))
        .andExpect(status().isOk()); // order placement is NOT affected by webhook failure

    // optionally: verify a WARN log line was emitted, via a log-capturing test appender
}
```

10.3 Manual/Exploratory Verification (for the demo)

- Place an order → observe the n8n execution log shows the order.allocated workflow firing with the correct payload.
- Manually stop the n8n container → place another order → confirm the API still returns 200 OK normally, and the order is fully persisted correctly.
- Send a request directly to the n8n webhook URL with a wrong/missing signature → confirm the workflow’s Function node rejects it before any notification fires.

11 Decision Note: Twilio Integration Point

The original design (main spec, Section 18) has `ReorderAlertJob` call `TwilioSmsClient` directly. With `n8n` in the picture, there are two valid options — **pick one and state the choice explicitly in the report, don't do both**, since duplicate SMS sends would be a real bug, not just an inelegance:

Option	Description	Recommendation
A: Keep direct Twilio call in Spring Boot	<code>ReorderAlertJob</code> still calls <code>TwilioSmsClient</code> directly for the reorder SMS; <code>stock.below_threshold</code> webhook only drives Slack/email/escalation via <code>n8n</code>	Simpler, recommended for the 3-week scope — fewer moving parts, and the SMS path stays inside the already-tested Spring Boot code
B: Move SMS entirely into <code>n8n</code>	<code>ReorderAlertJob</code> never calls Twilio; <code>n8n</code> 's native Twilio node handles SMS as part of Workflow 2	Cleaner separation of concerns (all notification logic lives in <code>n8n</code>), but adds <code>n8n</code> as a hard dependency for a previously-working feature, and is better suited as a Future Work item once the core demo is stable

State this decision explicitly in the project report's design section — an evaluator seeing an explicit “we considered both and chose A because X” is a stronger signal than either option implemented silently without acknowledgment of the alternative.

12 Updated Build Plan Addendum

Insert after Week 3, Day 19 of the main spec's build plan (Section 21), as a Days 22-24 extension if time allows, or fold into Day 20's dashboard slot if time is tight:

Day	Focus
22	WebhookEventPublisher, OrderAllocatedEvent/etc., HMAC signing utility, unit tests
23	n8n container added to Docker Compose; build Workflow 1 (order.allocated) and Workflow 2 (stock.below_threshold) end-to-end
24	Remaining workflows (shipped, cancelled, backordered), failure-isolation integration test, final demo rehearsal showing an order triggering a visible n8n execution

If time is genuinely tight, implementing **only Workflow 1 and Workflow 2** fully, with the other three left as documented-but-not-built extensions in the report, is a reasonable and honest scope cut — better than a rushed, broken implementation of all five.